

Programmazione 1

Corso c

27/10/2023
> La ricorsione (1)

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica
Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

- Introduzione
- Fattoriale
- Classici
- Fibonacci
- Ricorsione vs. Iterazione

Outline

- Introduzione
- Fattoriale
- Classici
- Fibonacci
- Ricorsione vs. Iterazione

Ricorsione

- Un programma C è composta da varie funzioni (almeno 1 main) che si chiamano l'un l'altra.
- Una **funzione ricorsiva** è una funzione che (ri)chiama se stessa direttamente o indirettamente attraverso un'altra funzione.
- La ricorsione è un argomento complesso che viene discusso a lungo nei corsi di informatica di livello superiore.

Ricorsione—Caso base e chiamata ricorsiva (1 of 2)

Gli approcci ricorsivi alla risoluzione dei problemi seguono uno schema (*pattern, template*) comune. Una funzione ricorsiva viene chiamata per risolvere un problema:

- La funzione sa come risolvere **solo il caso base** (o i casi base). **Se** la funzione viene richiamata su un caso base, restituisce semplicemente il risultato
- **Altrimenti**, se la funzione viene chiamata con un problema più complesso, la funzione divide il problema in due parti concettuali:
 1. uno che la funzione sa come fare e
 2. uno che la funzione non sa fare.

Per rendere effettiva la ricorsione, il punto 2 deve assomigliare al problema originale, ma deve essere una versione leggermente più semplice o più piccola.

Ricorsione—Caso base e chiamata ricorsiva (2 of 2)

- Questo nuovo problema assomiglia all'originale, quindi la funzione richiama se stessa: questa operazione viene definita **chiamata ricorsiva** o **passo di ricorsione**.
- Il passo di ricorsione include anche un return, e il suo risultato sarà combinato con la parte del problema che la funzione sapeva come risolvere per formare un risultato complessivo che sarà passato alla chiamante.
- Il passo di ricorsione viene eseguito mentre la chiamata originale alla funzione è in pausa, in attesa del risultato del passo di ricorsione
- La fase di ricorsione può dare luogo a molte altre chiamate ricorsive, poiché la funzione continua a dividere ogni problema con cui viene chiamata in due parti concettuali.
- Affinché la ricorsione termini, la sequenza di problemi più piccoli deve infine **convergere sul caso base**

Outline

- Introduzione
- Fattoriale
- Classici
- Fibonacci
- Ricorsione vs. Iterazione

Ricorsione - Fattoriale

- Il fattoriale di un intero non negativo n , scritto $n!$, è:

$$n! = n * (n-1) * (n-2) * \dots * 1$$

- $5! = 5 * 4 * 3 * 2 * 1$

- Possiamo calcolarlo ricorsivamente?

- Iterativamente:

```
int fattIte (int n) {  
    int ret = 1; //0!==1  
  
    for(int i=1; i<=n; i++)  
    {  
        ret = ret * i;  
    }  
    return ret;  
}
```


Ricorsione - Fattoriale

Il calcolo del fattoriale è *naturalmente* ricorsivo:

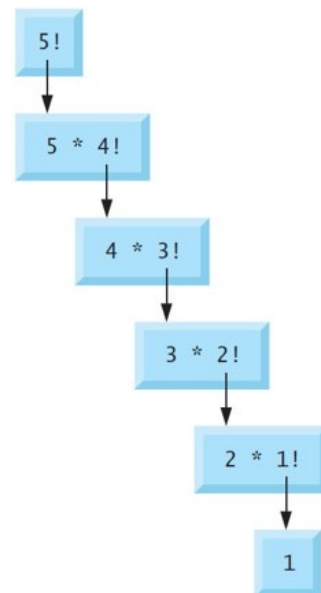
- $5! = 5 * 4 * 3 * 2 * 1 = 5 * (4 * 3 * 2 * 1) = 5 * 4!$
- $4! = 4 * (3 * 2 * 1) = 4 * 3!$
- $3! = 3 * (2 * 1) = 3 * 2!$
- $2! = 2 * (1) = 2 * 1!$
- $1! = 1 * (1) = 1 * 0!$
- $0! = 1$

$$n! = n * (n-1)!$$

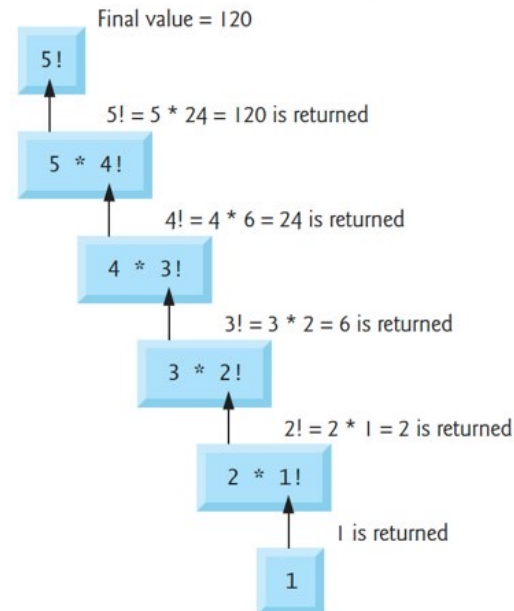
5! (libro: caso base 1!)

- La valutazione di 5! procederebbe come mostrato nel diagramma:
 - La parte (a) mostra come procede la successione di chiamate ricorsive fino a quando 1! viene valutato come 1 (cioè il caso base), terminando la ricorsione.
 - La parte (b) mostra i valori restituiti da ogni chiamata ricorsiva al suo chiamante, finché non viene calcolato e restituito il valore finale.

a) Sequence of recursive calls



b) Values returned from each recursive call



Programmazione ricorsiva

Una implementazione ricorsiva di `fattRic(n)` dovrà verificare il valore di `n`:

- Se `n == 0`, la funzione semplicemente ritorna 1 al chiamante
- Altrimenti (`n > 0`), la funzione dovrà:
 - delegare a `fatt(n-1)` il calcolo di `n-1!`, ovvero `fattRic(n)` chiama `fattRic(n-1)`
 - ritornare al chiamante `n * (n-1)!`

Fattoriale ricorsivo

- La funzione fattoriale è per definizione composta da un caso *base* e un caso *generale*

$$n! = \begin{cases} 1 & \text{se } n == 0 \\ n * (n-1)! & \text{altrimenti} \end{cases}$$

```
void main(void) {  
    int t = 3;  
    int res = fattRic(t);  
    printf("%d!: %d\n", t, res);  
}
```

```
int fattRic(int n) {  
    int ret = 0;  
    if (n == 0) {  
        ret = 1;  
    }  
    else {  
        ret = n * (n-1)!;  
    }  
    return ret;  
}
```

Fattoriale ricorsivo

- La funzione fattoriale è per definizione composta da un caso *base* e un caso *generale*

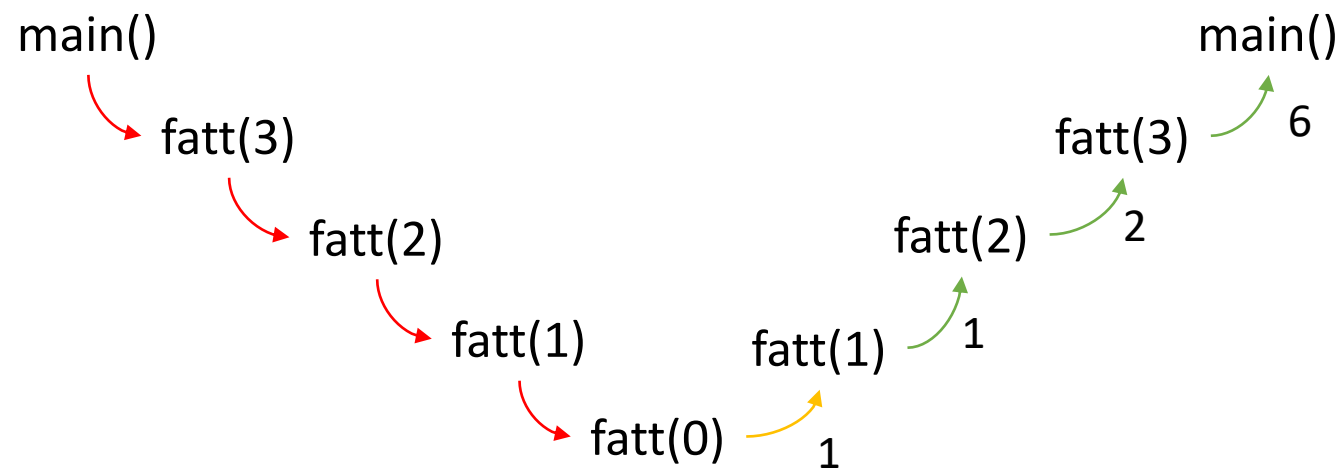
$$n! = \begin{cases} 1 & \text{se } n == 0 \\ n * (n-1)! & \text{altrimenti} \end{cases}$$

```
void main(void) {  
    int t = 3;  
    int res = fattRic(t);  
    printf("%d!: %d\n", t, res);  
}
```

```
int fattRic(int n) {  
    int ret = 0;  
    if (n == 0) {  
        ret = 1;  
    }  
    else {  
        ret = n * fattRic(n-1);  
    }  
    return ret;  
}
```

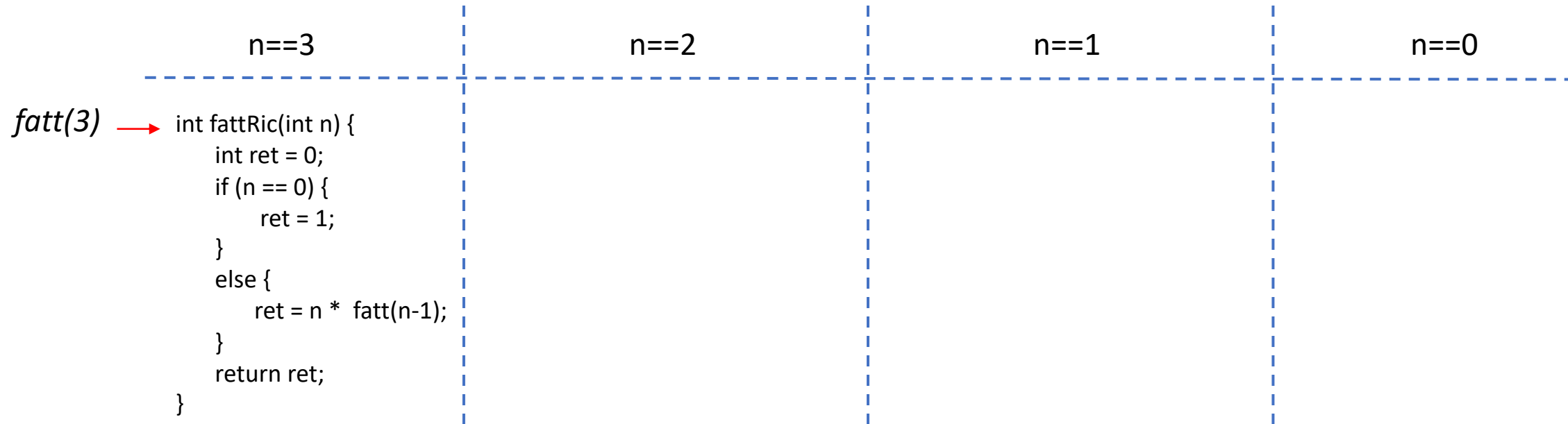
Fattoriale ricorsivo

- La chiamata a funzione avviene per n decrescenti
- Il calcolo di n! avviene per n crescenti al ritorno della ricorsione

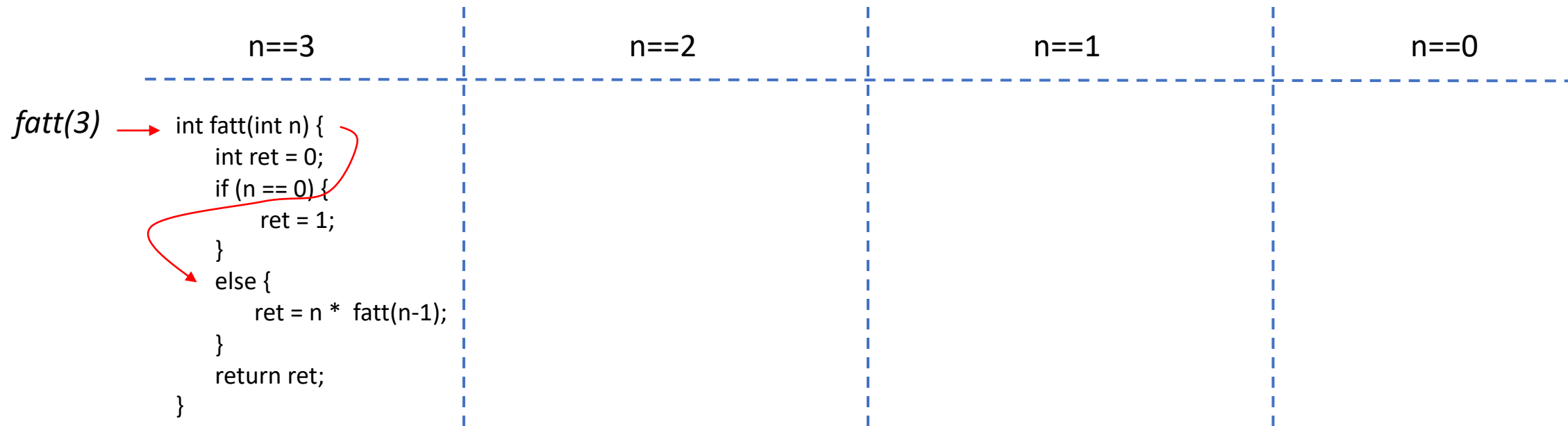


```
int fattRic(int n) {  
    int ret = 0;  
    if (n == 0) {  
        ret = 1;  
    }  
    else {  
        ret = n*fattRic(n-1);  
    }  
    return ret;  
}
```

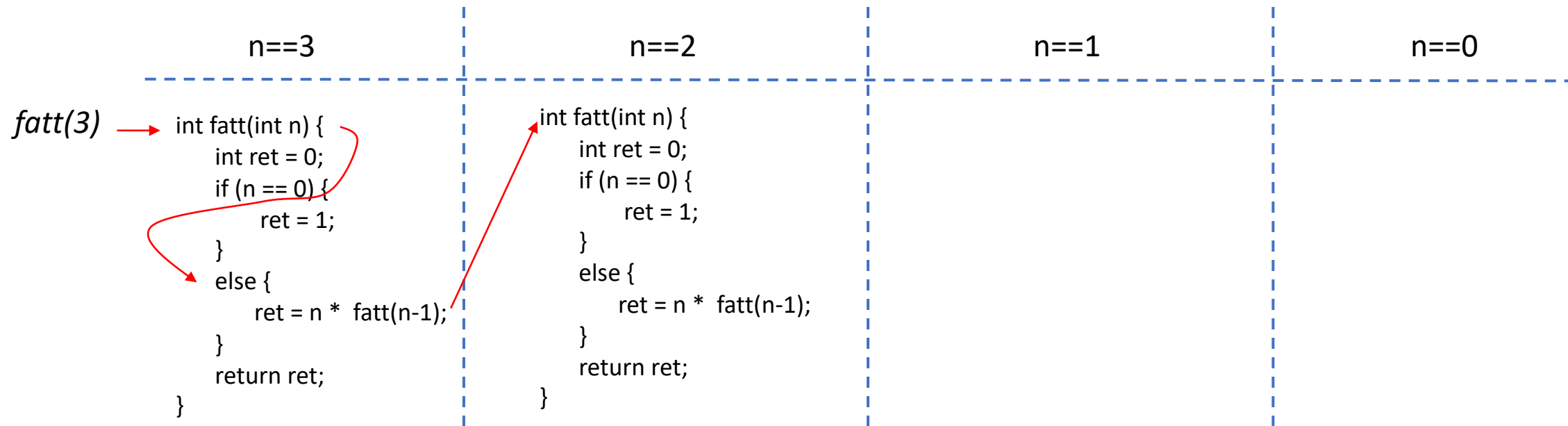
Fattoriale ricorsivo visualizzato



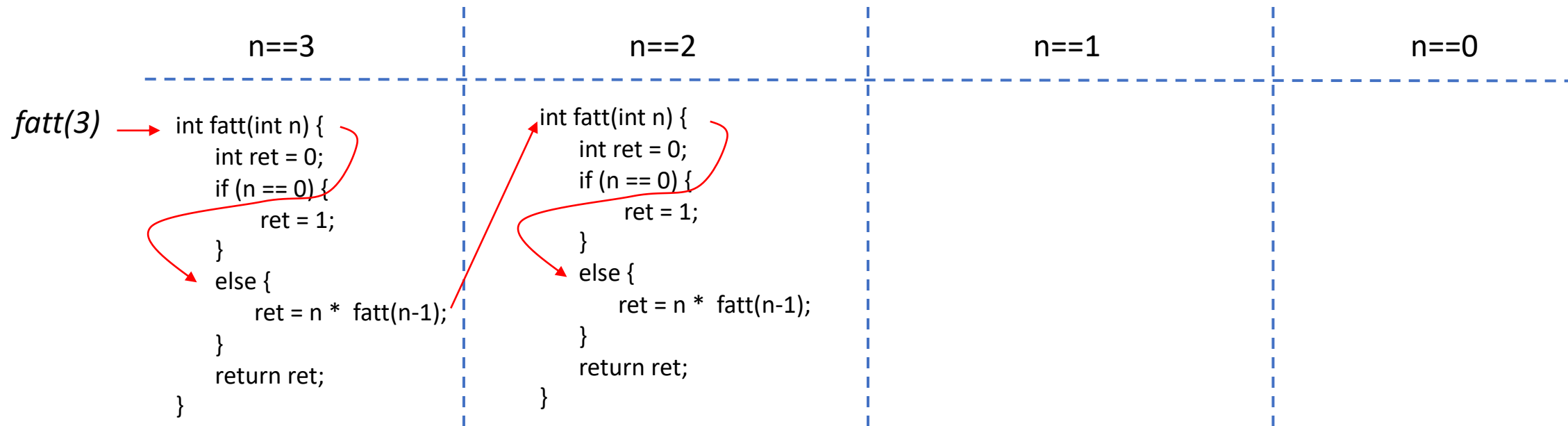
Fattoriale ricorsivo visualizzato



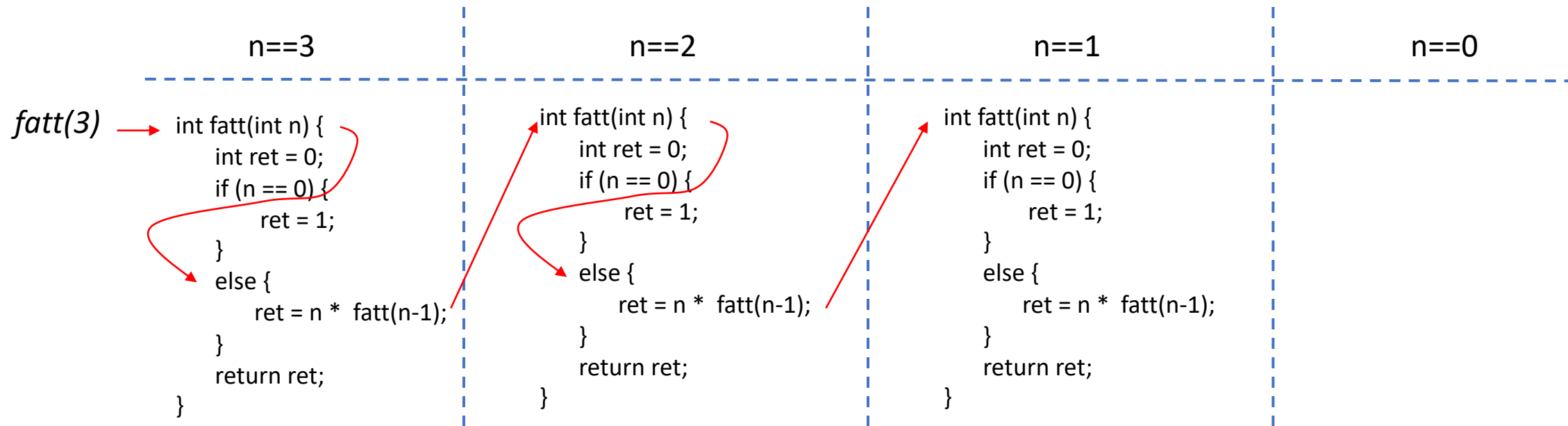
Fattoriale ricorsivo visualizzato



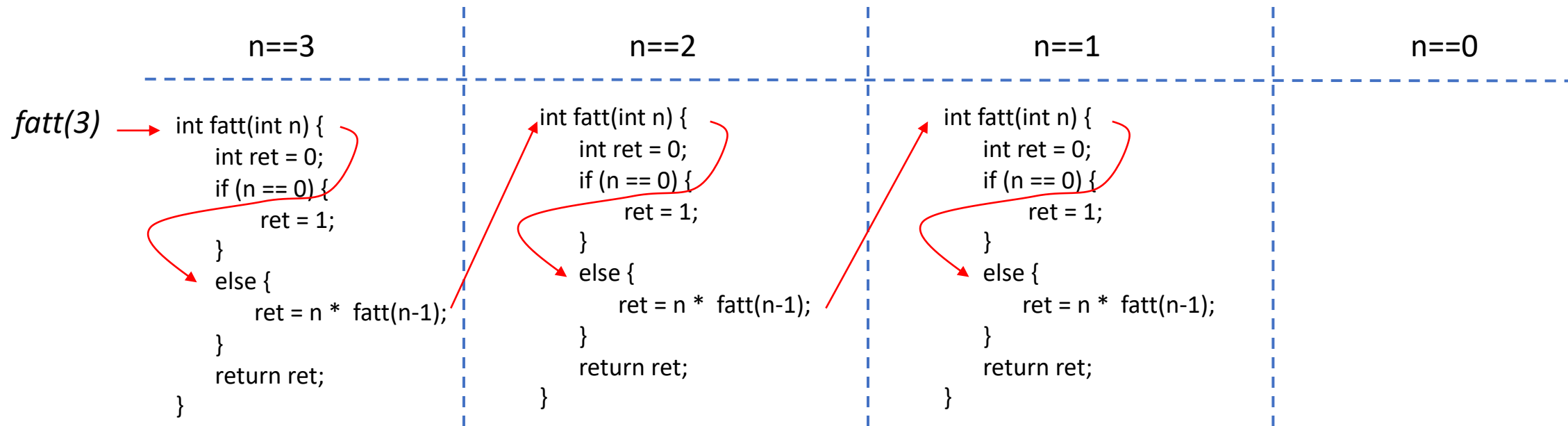
Fattoriale ricorsivo visualizzato



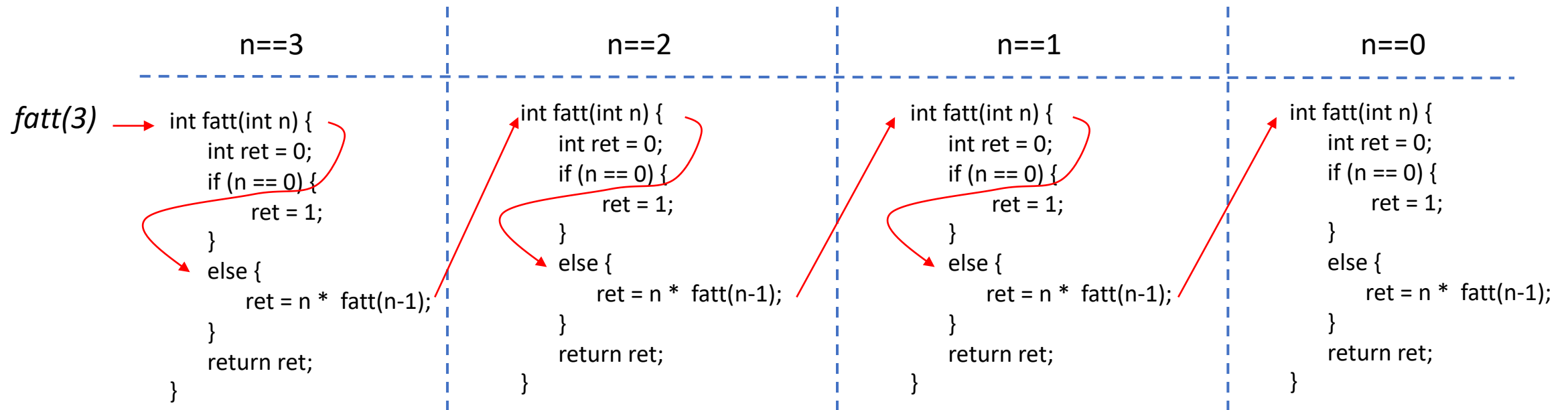
Fattoriale ricorsivo visualizzato



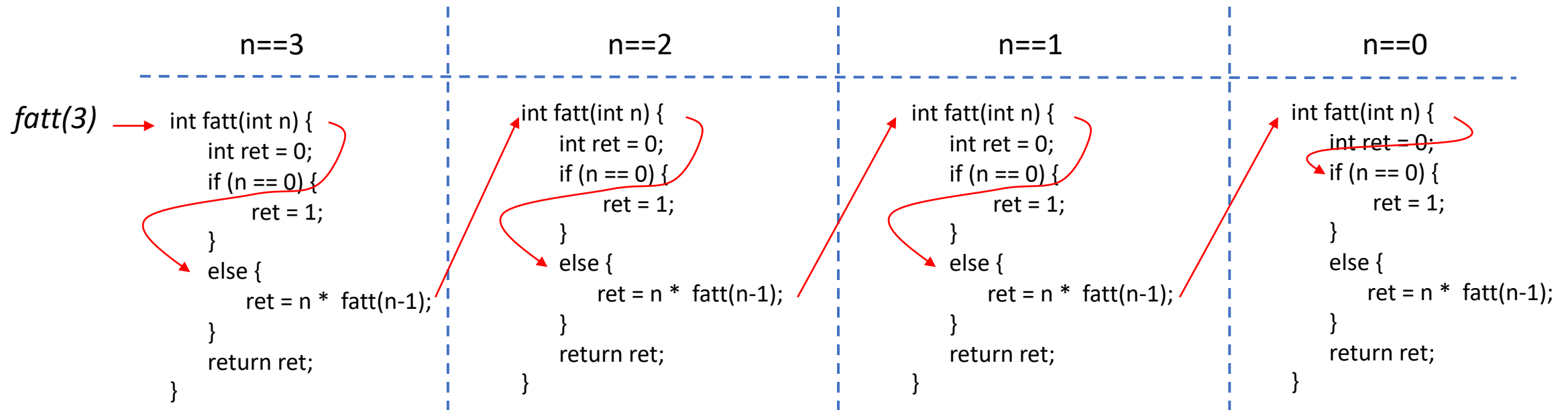
Fattoriale ricorsivo visualizzato



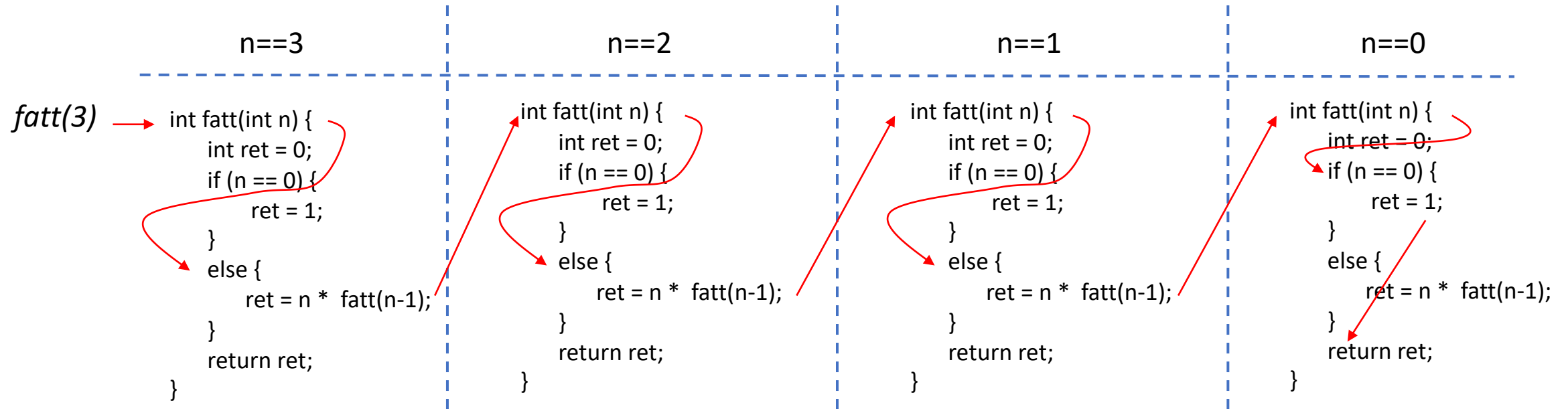
Fattoriale ricorsivo visualizzato



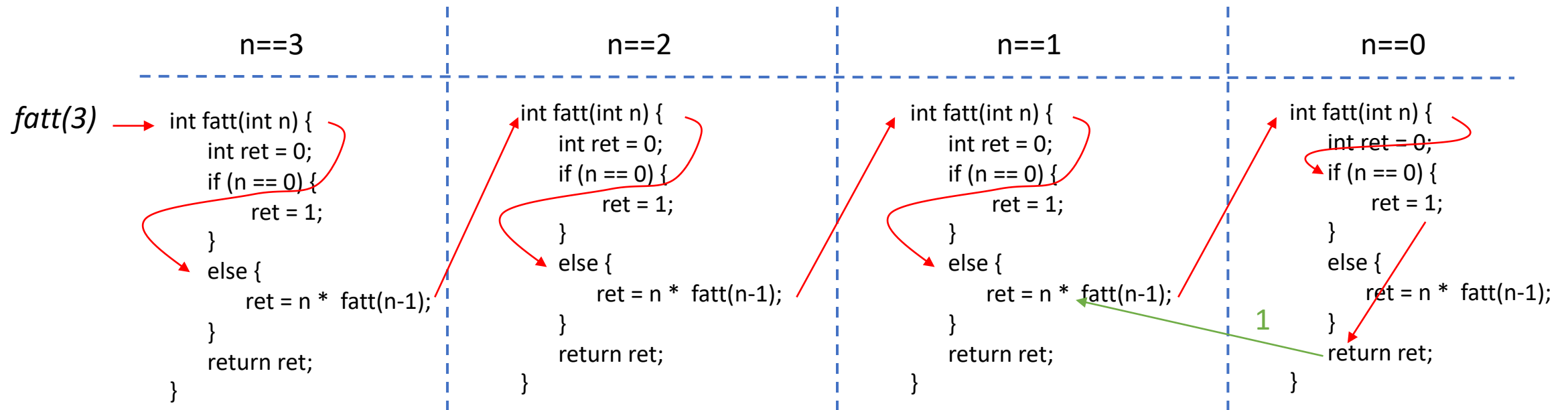
Fattoriale ricorsivo visualizzato



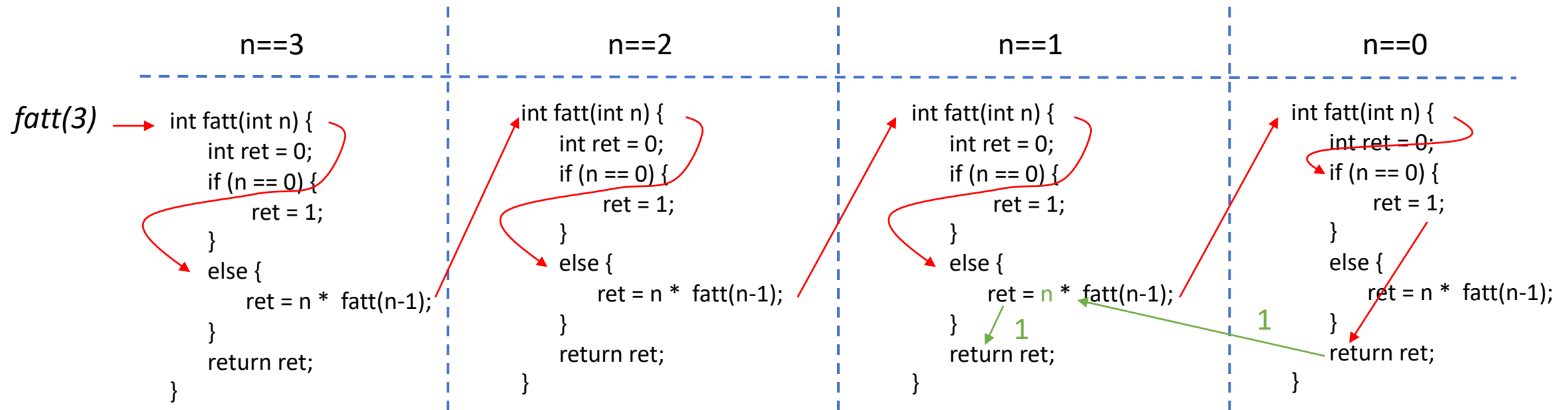
Fattoriale ricorsivo visualizzato



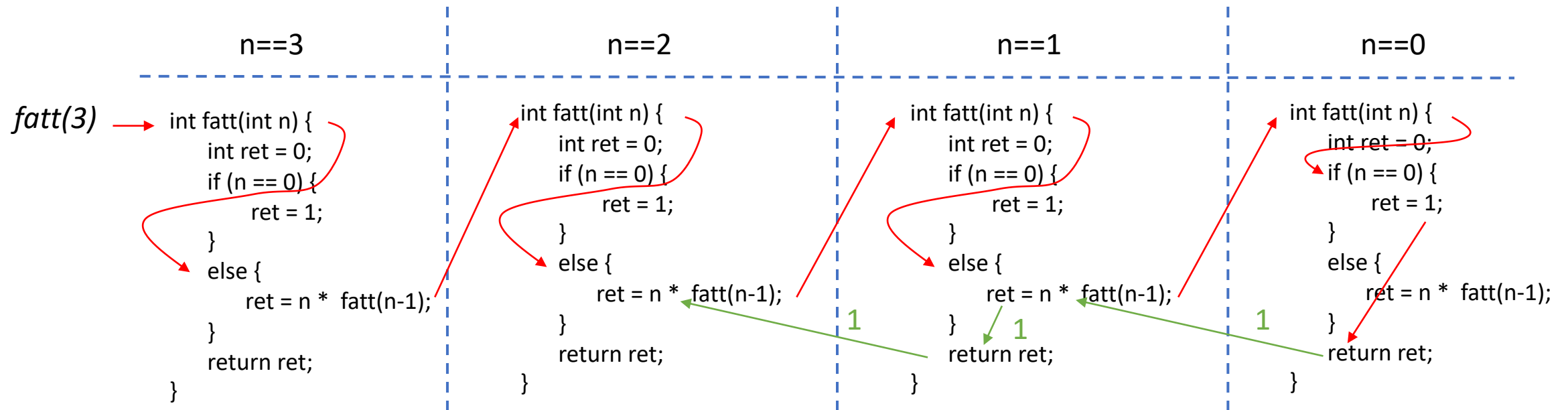
Fattoriale ricorsivo visualizzato



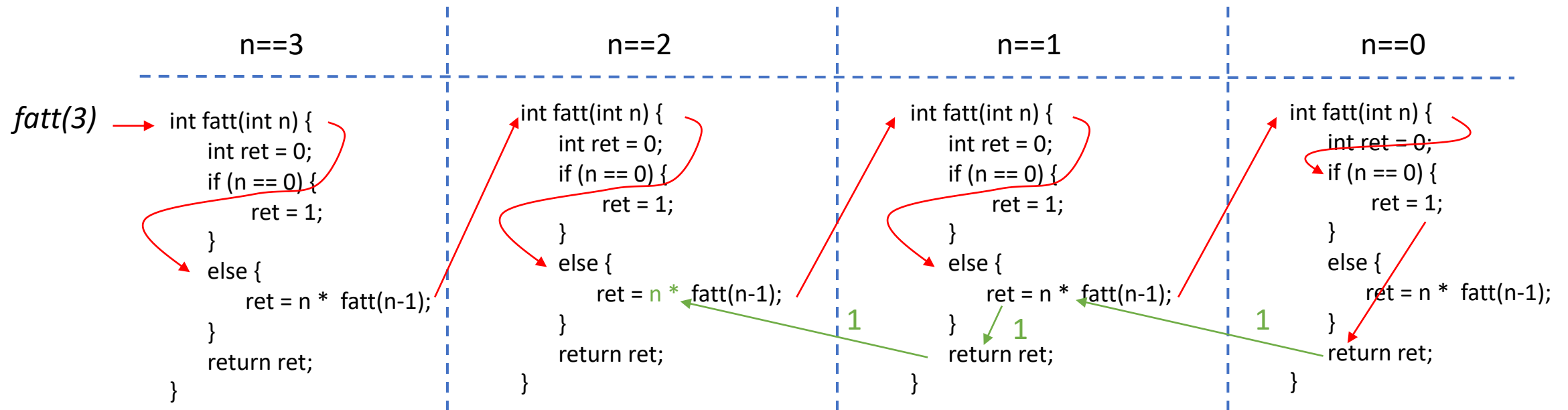
Fattoriale ricorsivo visualizzato



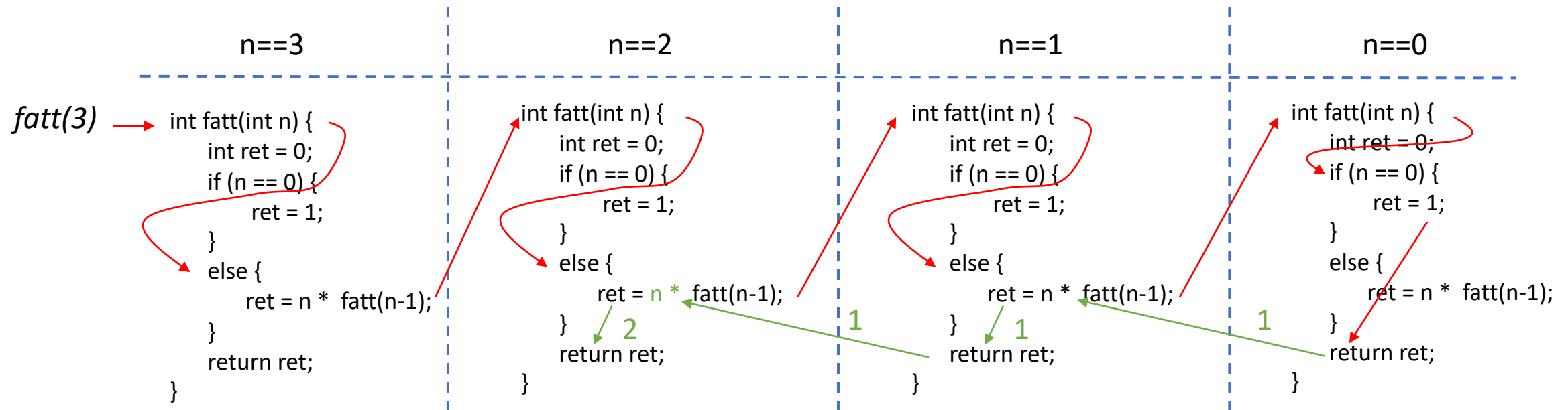
Fattoriale ricorsivo visualizzato



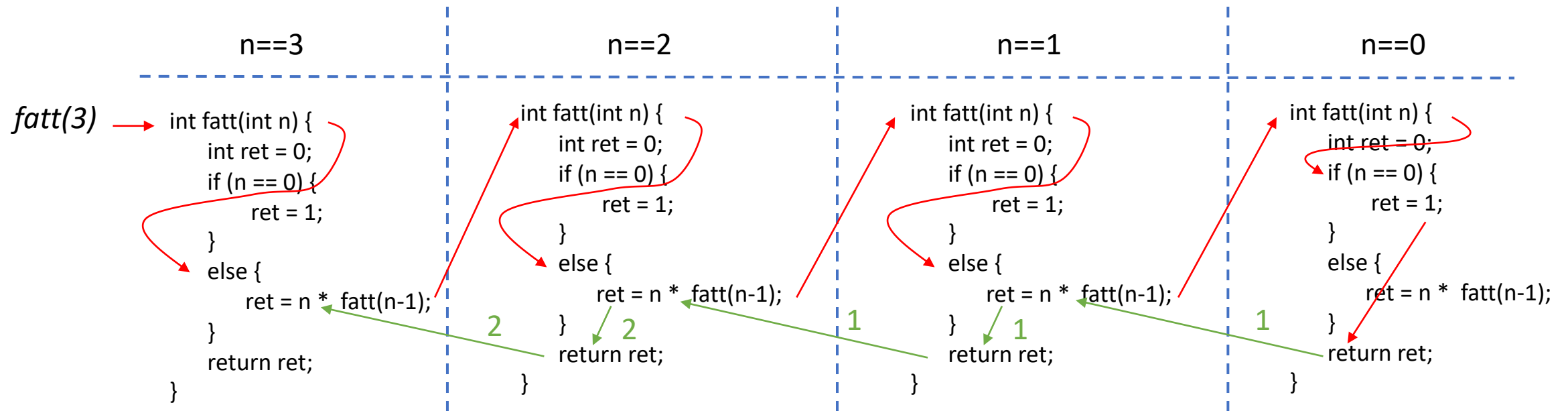
Fattoriale ricorsivo visualizzato



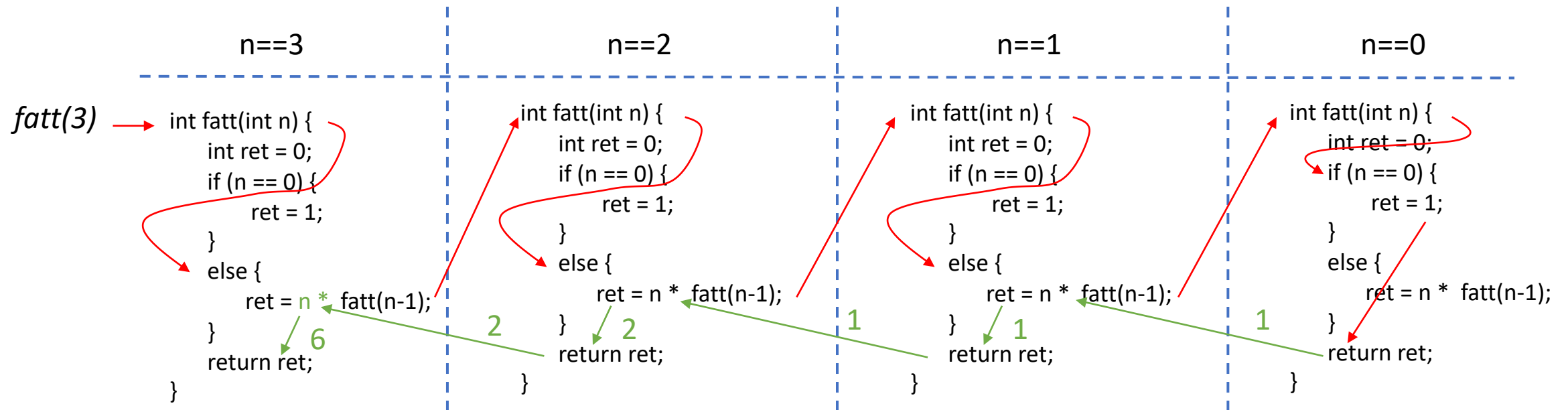
Fattoriale ricorsivo visualizzato



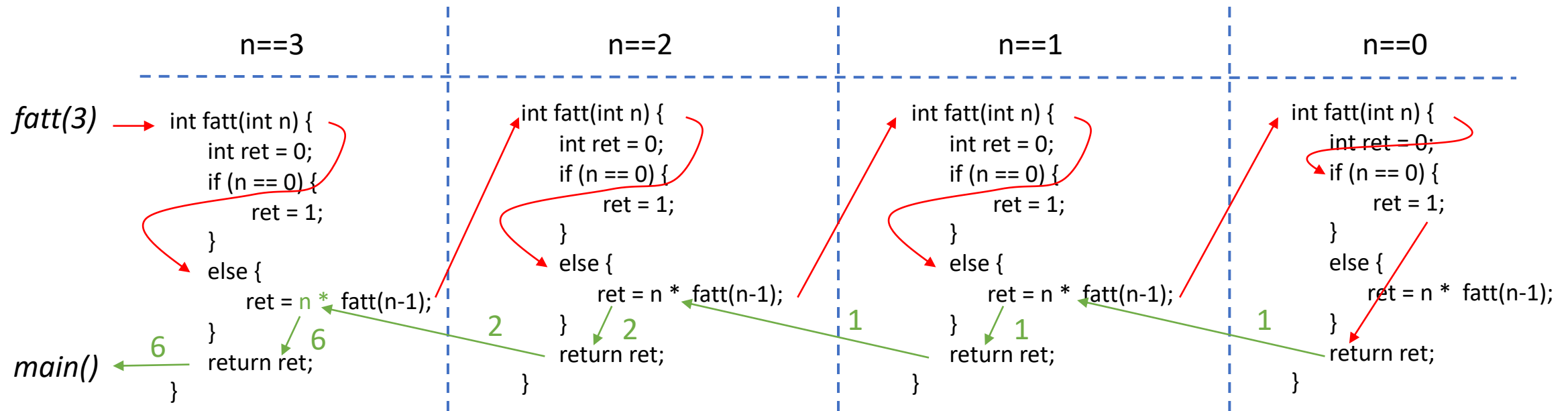
Fattoriale ricorsivo visualizzato



Fattoriale ricorsivo visualizzato



Fattoriale ricorsivo visualizzato



5.14 Ricorsione: implementazione de fattoriale ricorsivo (1 of 4)

- La Figura 5.9 utilizza la ricorsione per calcolare e stampare i fattoriali dei numeri interi 0-21.

```
1. // fig05_09.c
2. // Recursive factorial function.
3. #include <stdio.h>
4.
5. unsigned long long int factorial(int number);
6.
7. int main(void) {
8.     // calculate factorial(i) and display result
9.     for (int i = 0; i <= 21; ++i) {
10.         printf("%d! = %llu\n", i, factorial(i));
11.     }
12. }
```


5.14 Ricorsione: implementazione de fattoriale ricorsivo (2 of 4)

```
13.  
14. // recursive definition of function factorial  
15. unsigned long long int factorial(int number) {  
16.     if (number <= 1) { // base case  
17.         return 1;  
18.     }  
19.     else { // recursive step  
20.         return (number * factorial(number - 1));  
21.     }  
22. }
```

5.14 Ricorsione: implementazione de fattoriale ricorsivo (3 of 4)

$$0! = 1$$

$$1! = 1$$

$$2! = 2$$

$$3! = 6$$

$$4! = 24$$

$$5! = 120$$

$$6! = 720$$

$$7! = 5040$$

$$8! = 40320$$

$$9! = 362880$$

$$10! = 3628800$$

5.14 Ricorsione: implementazione de fattoriale ricorsivo (4 of 4)

11! = 39916800

12! = 479001600

13! = 6227020800

14! = 87178291200

15! = 1307674368000

16! = 20922789888000

17! = 355687428096000

18! = 6402373705728000

19! = 121645100408832000

20! = 2432902008176640000

21! = 14197454024290336768

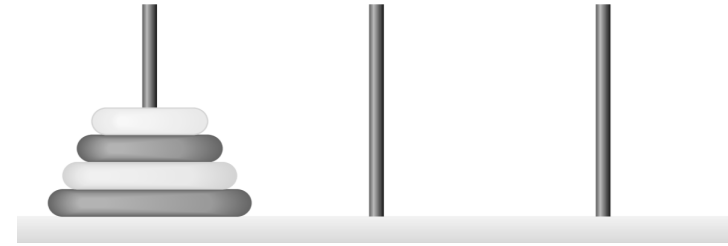
5.14 La ricorsione - il fattoriale diventa grande velocemente

- `factorial` ritorna un `unsigned long long int`
- Può contenere un valore grande almeno quanto 18.446.744.073.709.551.615
- I valori del fattoriale diventano grandi rapidamente
- Si è usato `unsigned long long int` in modo che il programma possa calcolare valori fattoriali più grandi
- Lo specificatore `%llu` è usato per stampare valori `unsigned long long int`
- `factorial` produce valori molto grandi rapidamente, anche il valore massimo di `unsigned long long int` è raggiunto rapidamente

Outline

- Introduzione
- Fattoriale
- Classici
- Fibonacci
- Ricorsione vs. Iterazione

Torre di Hanoi



La leggenda narra che in un tempio dell'Estremo Oriente alcuni sacerdoti tentano di spostare una pila di dischi da un piolo a un altro. La pila iniziale aveva 64 dischi infilati in un piolo, disposti in dimensione decrescente dal basso verso l'alto. I sacerdoti tentano di spostare la pila da questo piolo a un secondo piolo, con il vincolo che si sposti esattamente un disco alla volta e che nessun disco più grande possa essere collocato sopra un disco più piccolo. Un terzo piolo è disponibile per contenere temporaneamente i dischi. Presumibilmente il mondo finirà quando i sacerdoti porteranno a termine il loro compito, così siamo poco incentivati a facilitare i loro sforzi.

Esempi di ricorsione (classici)

- Sommatoria $S(n)$
- Moltiplicare due interi
- Funzione potenza n^m
- MCD Euclide

Outline

- Introduzione
- Fattoriale
- Classici
- Fibonacci
- Ricorsione vs. Iterazione

5.15 Esempio di ricorsione: Serie di Fibonacci (1 di 5)

- La serie di Fibonacci inizia con 0 e 1 e ha la proprietà che ogni numero di Fibonacci successivo è la somma dei due numeri di Fibonacci precedenti: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
- Definizione ricorsiva:
 $\text{fibonacci}(0) = 0$
 $\text{fibonacci}(1) = 1$
 $\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$
- Compare in natura è forma una spirale (anche sulla mole!). Il rapporto tra i numeri di Fibonacci successivi converge verso un valore costante di 1,618... Questo numero si ripete in natura ed è chiamato rapporto aureo o media aurea. Gli esseri umani tendono a trovare la media aurea esteticamente gradevole, per cui viene spesso utilizzata nel design

5.15 Esempio di ricorsione: Serie di Fibonacci (2 di 5)

```
1. // fig05_10.c
2. // Recursive fibonacci function.
3. #include <stdio.h>
4.
5. unsigned long long int fibonacci(int n); // function prototype
6.
7. int main(void) {
8.     // calculate and display fibonacci(number) for 0-10
9.     for (int number = 0; number <= 10; number++) {
10.         printf("Fibonacci(%d) = %llu\n", number, fibonacci(number));
11.     }
12.
13.     printf("Fibonacci(20) = %llu\n", fibonacci(20));
14.     printf("Fibonacci(30) = %llu\n", fibonacci(30));
15.     printf("Fibonacci(40) = %llu\n", fibonacci(40));
16. }
```

5.15 Esempio di ricorsione: Serie di Fibonacci (3 di 5)

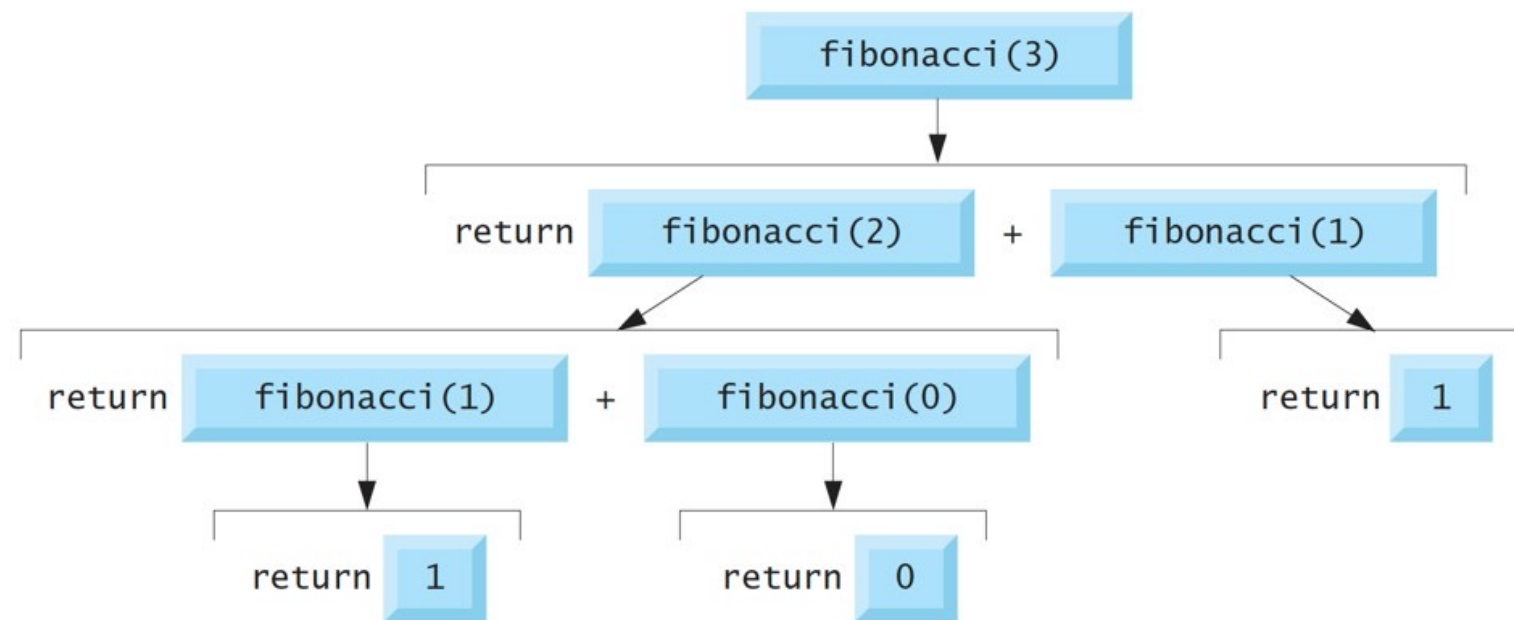
```
17.  
18. // Recursive definition of function fibonacci  
19. unsigned long long int fibonacci(int n) {  
20.     if (0 == n || 1 == n) { // base case  
21.         return n;  
22.     }  
23.     else { // recursive step  
24.         return fibonacci(n - 1) + fibonacci(n - 2);  
25.     }  
26. }
```

5.15 Esempio di ricorsione: Serie di Fibonacci (4 di 5)

- Le chiamate a `fibonacci` dal `main` **non** sono ricorsive (linee 10 e 13–15), ma tutte le seguenti chiamate a `fibonacci` sono ricorsive (linea 24).
- Ogni volta `fibonacci` è invocata verifica immediatamente che il **caso base**, cioè **`n` è uguale a 0 o 1**. Se questo è vero, `n` è ritornato
- Se `n` è più grande di 1, il passo ricorsivo genera **due chiamate ricorsive**, ognuna delle quali rappresenta un problema leggermente più semplice rispetto alla chiamata originale a `fibonacci`.

5.15 Esempio di ricorsione: Serie di Fibonacci (5 di 5)

- Diagramma che mostra come la funzione `fibonacci` valuta `fibonacci(3)`



5.15 Serie di Fibonacci: Ordine di valutazione degli operandi

- La funzione ricorsiva `fibonacci` pone alcune domande sull'**ordine** in cui i compilatori C valutano gli operandi degli operatori. Questo è un problema diverso dall'ordine in cui gli operatori vengono applicati ai loro operandi, dettato dalla precedenza e dal raggruppamento degli operatori (es. $2*3+4$).
- Nella valutazione di `fibonacci(3)`, **due** chiamate ricorsive sono fatte, cioè `fibonacci(2)` e `fibonacci(1)`
- Ma in quale ordine verranno effettuate queste chiamate?
- Per ragioni di ottimizzazione, il C non specifica l'ordine in cui devono essere valutati gli operandi della maggior parte degli operatori (compreso il `+`).
- In questo e nella maggior parte degli altri programmi, il risultato finale non dipende (e NON DEVE DIPENDERE) dall'ordine di valutazione. Se un operando ha **effetti collaterali (side effects)**, ciò potrebbe influenzare il risultato finale di un'espressione.

5.15 Operatori per i quali è specificato l'ordine di valutazione degli operandi

- Il C specifica l'ordine di valutazione degli operandi solo per &&, ||, l'operatore virgola "," e ?:
- I primi tre sono operatori binari i cui operandi sono garantiti per essere valutati **left-to-right**
 - **ATTENZIONE:** "," non è quella che separa gli argomenti di una funzione.
- `op1?op2:op3` è l'unico operatore ternario del C
 - Il suo operando più a sinistra viene sempre valutato per primo
 - Se vero, l'operando centrale viene valutato per primo e l'ultimo operando viene ignorato.
 - Se falso, il terzo operando viene valutato per primo e l'operando centrale viene ignorato.

5.15 Esempio di ricorsione: Serie di Fibonacci - Complessità esponenziale

- Attenzione alle funzioni ricorsive come Fibonacci: Ogni livello di ricorsione ha un effetto di raddoppio del numero di chiamate
- Il numero di chiamate ricorsive per calcolare l'ennesimo numero di Fibonacci è dell'ordine di 2^n . Il calcolo del 20° numero di Fibonacci richiede circa un milione di chiamate 2^{20} . Il calcolo del 30° numero di Fibonacci richiede circa un miliardo di chiamate 2^{30} .
- Gli informatici chiamano questo fenomeno "complessità esponenziale". Può mettere in crisi anche i computer più potenti del mondo!

Outline

- Introduzione
- Fattoriale
- Classici
- Fibonacci
- Ricorsione vs. Iterazione

prof. Roversi su Iterazione vs Ricorsione

- Da un lato, il principio guida alla base della programmazione iterativa è che essa costruisce un risultato, accumulando, passo dopo passo, sue approssimazioni sempre migliori.
- Dall'altro, la programmazione ricorsiva assume una visione specifica dello schema *divide et impera*:
 - Dato un problema, lo si scompone in modo che lo stesso problema si ripresenti, ma in situazioni più semplici di quello iniziale da risolvere.
 - Una volta ottenute le soluzioni alle versioni più semplici, si ricompongono (opportunamente) per ottenere la soluzione alla versione iniziale del problema

Fattoriale Iterativo e ricorsivo

```
int fattIte (int n) {  
    int ret = 1;  
    for(int i=1; i<=n; i++)  
    {  
        ret = ret * i;  
    }  
    return ret;  
}
```

```
int fattRic(int n) {  
    int ret = 0;  
    if (n == 0) {  
        ret = 1;  
    }  
    else {  
        ret = n*fattRic(n-1);  
    }  
    return ret;  
}
```

5.16 Ricorsione versus iterazione (1di 2)

Caratteristiche comuni di iterazione e ricorsione

- Entrambe si basano su un'istruzione di **controllo**
 - L'iterazione utilizza **un'istruzione di iterazione**; la ricorsione utilizza **un'istruzione di selezione**.
- Entrambi prevedono la **ripetizione**
 - L'iterazione utilizza **un'istruzione di iterazione**; la ricorsione utilizza **chiamate a funzione ripetute**.
- Entrambi hanno un **test di terminazione**
 - L'iterazione utilizza la **condizione di continuazione del ciclo che fallisce**; la ricorsione **raggiunge un caso base**.

5.16 Ricorsione versus iterazione (2di 2)

Caratteristiche comuni di iterazione e ricorsione

- L'iterazione controllata da un contatore e la ricorsione si avvicinano entrambe **gradualmente alla terminazione.**
 - L'iterazione modifica un contatore finché non **fallisce la condizione di interruzione del ciclo**; la ricorsione produce versioni più semplici del problema originale finché non viene **raggiunto il caso base.**
- Entrambi possono andare ***in loop***
 - Un **ciclo infinito** si verifica se il test di interruzione del ciclo non diventa **mai falso**; la **ricorsione infinita** si verifica se il passo di ricorsione **non converge sul caso base.**

5.16 Ricorsione versus iterazione: problemi della ricorsione

- **Richiama** ripetutamente il meccanismo, e di conseguenza **l'overhead**, delle chiamate di funzione.
- Può essere costoso sia in termini di tempo del processore che di spazio di memoria.
- Ogni chiamata ricorsiva causa la creazione di un'altra copia della funzione (in realtà solo delle variabili della funzione nel frame). Può consumare **molta memoria**
- L'iterazione, invece, avviene normalmente all'interno di una funzione, quindi l'overhead delle chiamate di funzione ripetute e l'assegnazione di memoria extra vengono omessi.

5.16 Ricorsione versus iterazione: la ricorsione NON è indispensabile

- Qualsiasi problema che può essere risolto in modo ricorsivo può essere risolto anche in modo iterativo (cioè non ricorsivo).
- Si sceglie un approccio ricorsivo quando **questo rispecchia in modo più naturale il problema** e risulta in un programma più facile da capire e da debuggare. N.B: **il problema o la struttura dati**.
- Un'altra ragione per scegliere una soluzione ricorsiva è che **una soluzione iterativa può non essere evidente**

5.16 Ricorsione versus iterazione: Osservazioni conclusive (1 di 2)

- È importante una buona ingegneria del software e sono importanti le prestazioni elevate.
- Purtroppo, questi obiettivi sono spesso in contrasto tra loro.
 - Una buona ingegneria del software è fondamentale per rendere più gestibile il compito di sviluppare i sistemi software più grandi e complessi di cui abbiamo bisogno.
 - Le alte prestazioni sono fondamentali per realizzare i sistemi del futuro, che richiederanno all'hardware requisiti di calcolo sempre più elevati.

5.16 Ricorsione versus iterazione:

Osservazioni conclusive (2 di 2)

- **Software Engineering**
 - La suddivisione di un programma di grandi dimensioni in funzioni favorisce una buona ingegneria del software. Ma ha un prezzo.
 - Un programma fortemente funzionalizzato effettua un numero potenzialmente elevato di chiamate a funzioni. Queste consumano tempo di esecuzione sui processori del computer.
- **Performance**
 - Le architetture hardware di oggi sono state ottimizzate per rendere efficienti le chiamate di funzione.
 - I compilatori C aiutano a ottimizzare il codice e i processori hardware e le architetture multicore di oggi sono incredibilmente veloci.
 - **Per la stragrande maggioranza delle applicazioni e dei sistemi software, una buona ingegneria del software sarà più importante della programmazione ad alte prestazioni.**
 - Tuttavia, in molte applicazioni e sistemi, come la programmazione di giochi, i sistemi in tempo reale, i sistemi operativi e i sistemi embedded, le prestazioni sono fondamentali

5.16 Ricorsione: esempi nel libro

Chapter 5

- Factorial function
- Fibonacci function
- Greatest common divisor
- Multiply two integers
- Raising an integer to an integer power
- Towers of Hanoi
- Recursive main
- Visualizing recursion

Chapter 6

- Sum the elements of an array
- Print an array
- Print an array backward
- Print a string backward
- Check whether a string is a palindrome
- Minimum value in an array
- Linear search
- Binary search
- Eight Queens

Chapter 7

- Maze traversal

Chapter 8

- Printing a string input at the keyboard backward

Chapter 12

- Search a linked list
- Print a linked list backward
- Binary tree insert
- Preorder traversal of a binary tree
- Inorder traversal of a binary tree
- Postorder traversal of a binary tree
- Printing trees

Chapter 13

- Selection sort
- Quicksort

Progettare algoritmi ricorsivi

- Nel caso del fattoriale, il problema è già espresso in forma adatta ad un'implementazione ricorsiva
- Prima di implementare un algoritmo ricorsivamente, è necessario identificare un caso base ed un caso generale
- Facciamo ancora un esempio su intero n per poi passare ad array e matrici

Stampa numeri crescente [0, ..., N]

- Stampare i numeri naturali nel segmento [0,...,N] in ordine crescente tramite una funzione

```
void stampaC(int n)
```

che sia implementata in forma ricorsiva ovvero senza far ricorso a cicli

- La funzione viene obbligatoriamente chiamata da `main()` come `stampaC(N)`
- Output atteso dalla funzione per `N=3`

0 1 2 3

Stampa numeri crescente $[0, \dots, N]$

- Occorre far scorrere l'indice della ricorsione all'indietro da N fino a 0 . Questo rappresenta il caso generale
- Arrivato a $n==0$, stampo 0 e ritorno dalla ricorsione. Questo è il caso base

Stampa numeri crescente [0, ..., N]

```
int main(void) {  
    stampaC (3);  
}  
  
void stampaC (int n) {  
    if (n == 0) {  
        printf("0 ");  
    }  
    else {  
        stampaC (n-1);  
        printf("%d ", n);  
    }  
    return;  
}
```

return esplicitato
per ragioni didattiche
(ritorna *void*)

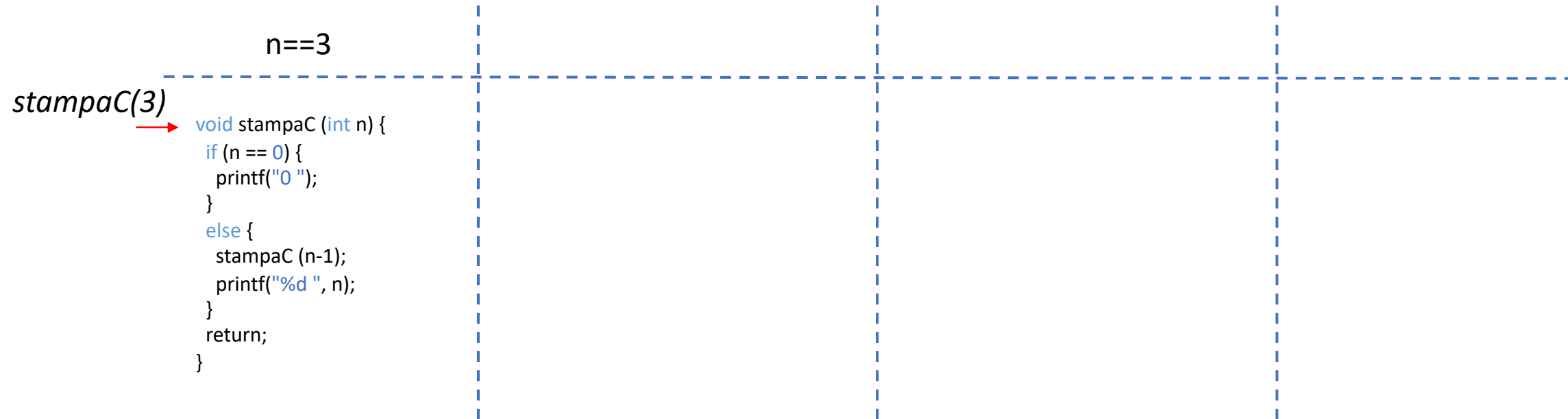


Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione

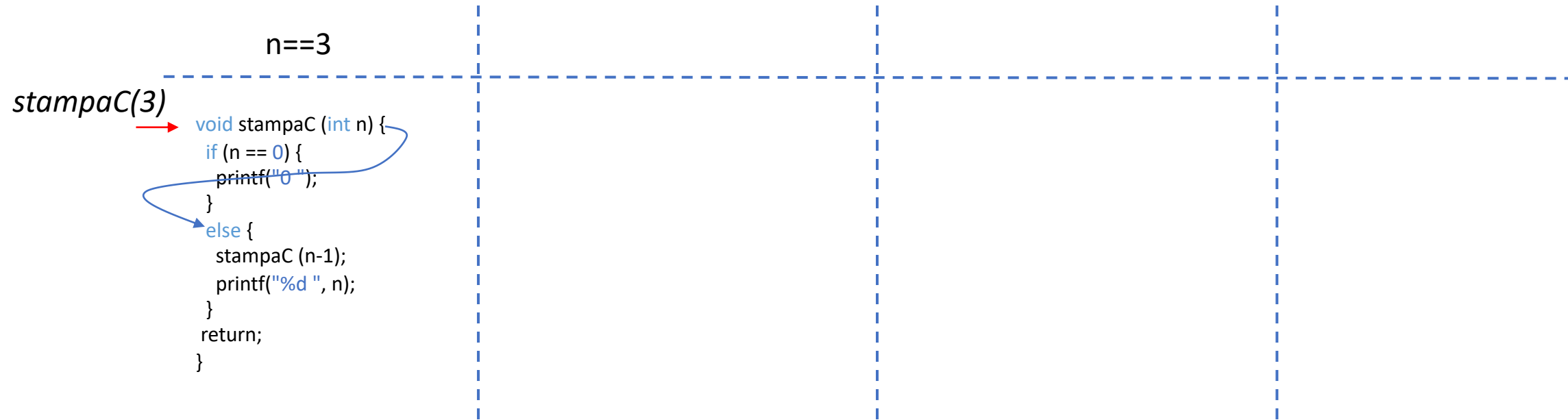


Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione

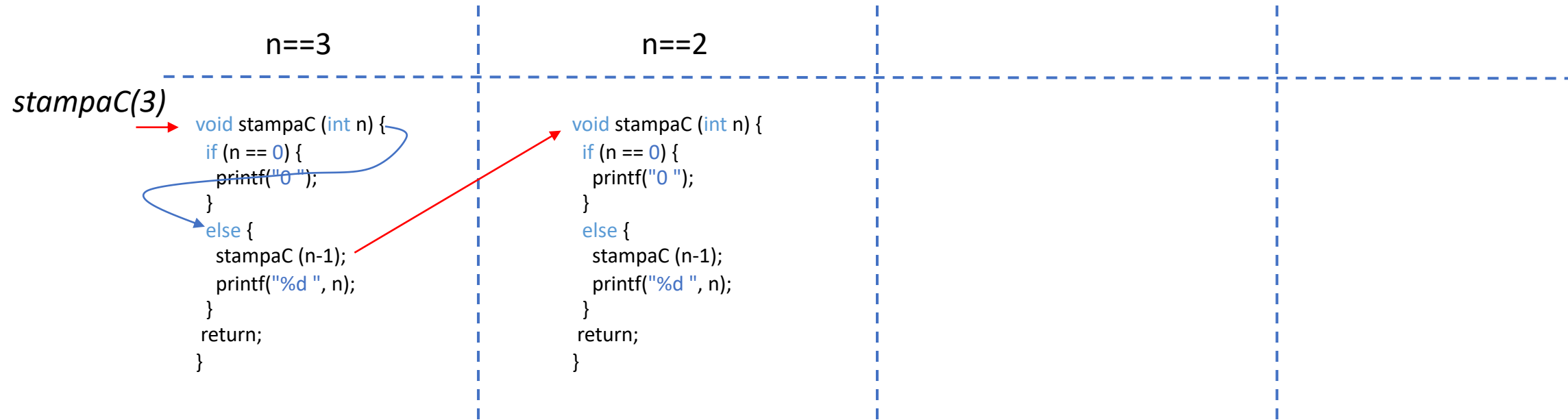


Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione



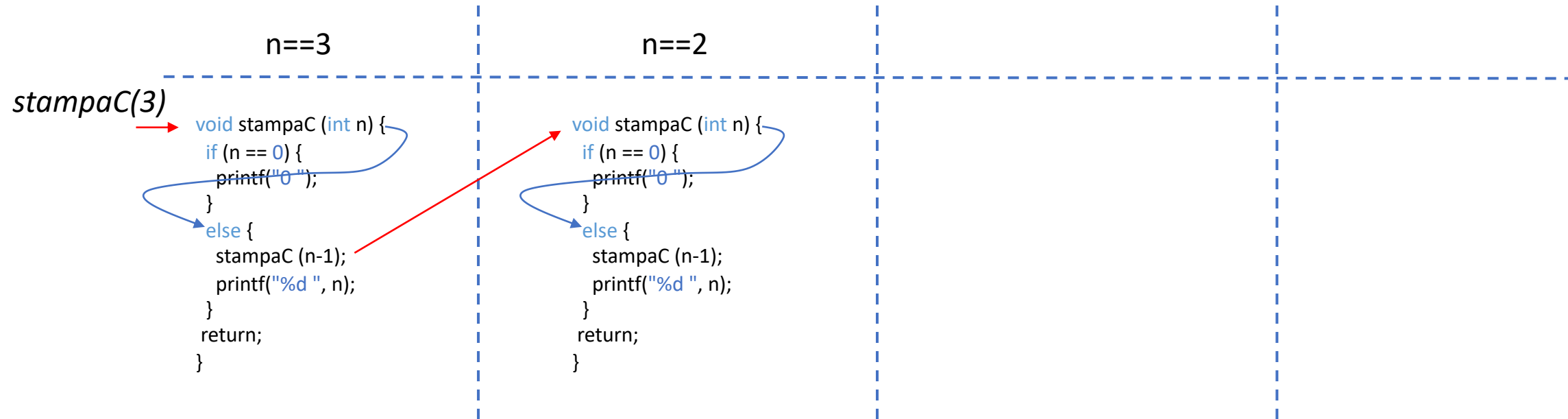
Chiamata a `stampaC(2)`

Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione

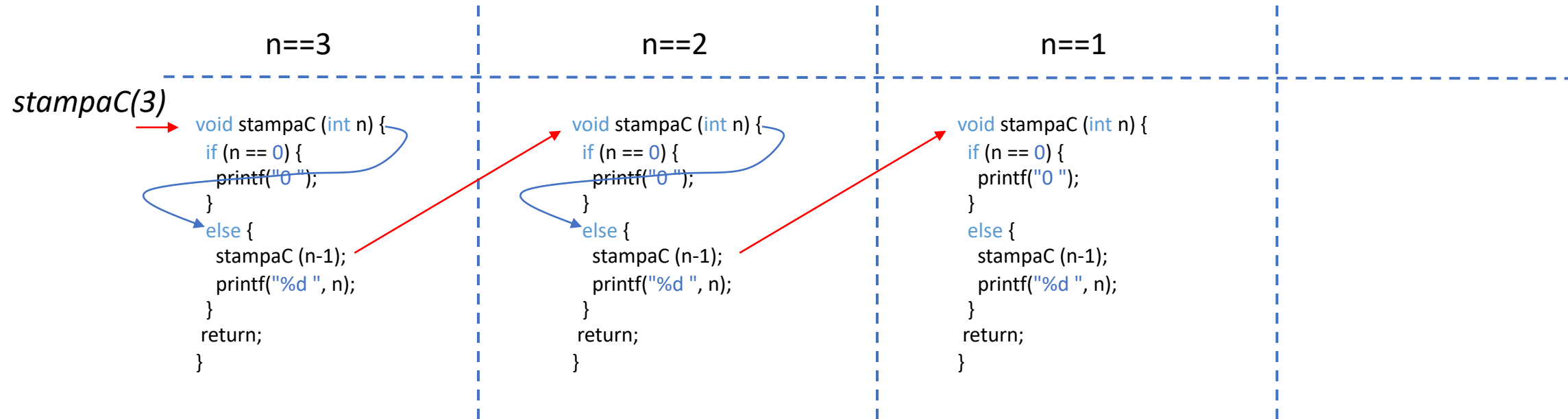


Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione



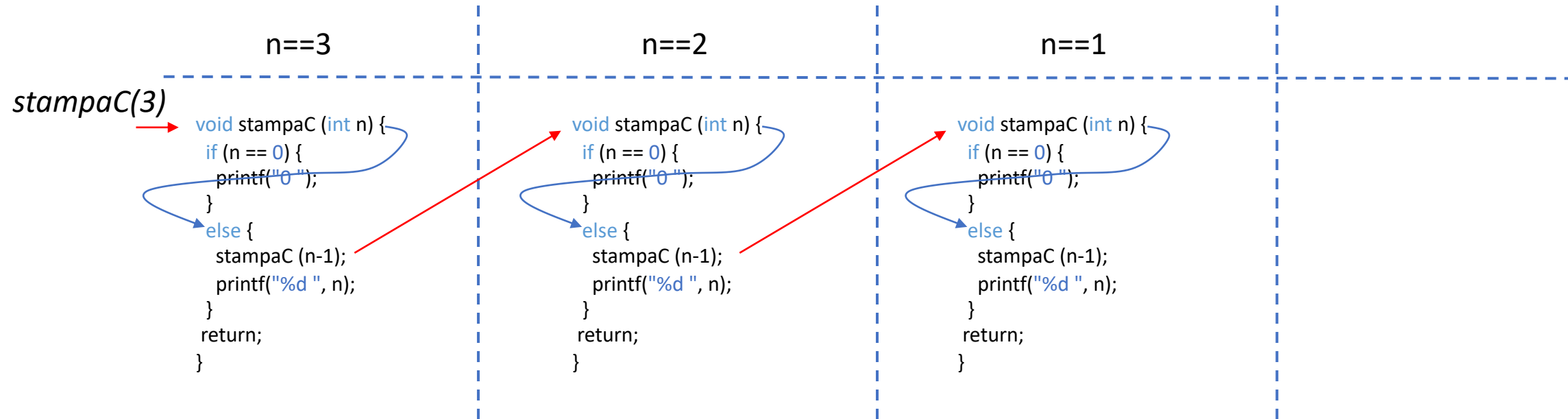
Chiamata a `stampaC(1)`

Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione

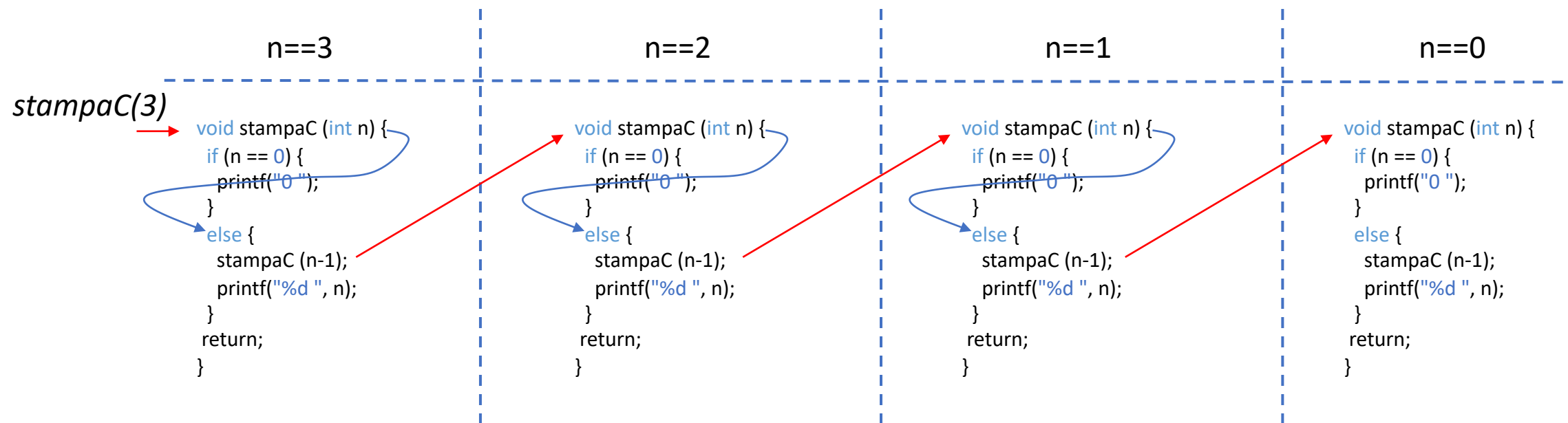


Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione



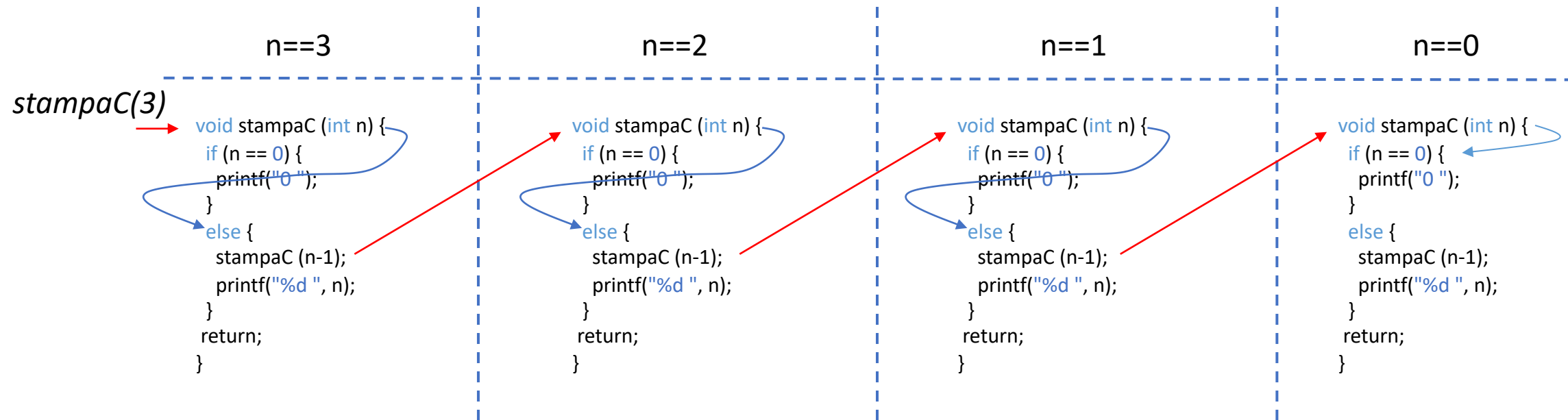
Chiamata a `stampaC(0)`

Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N  
0
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione

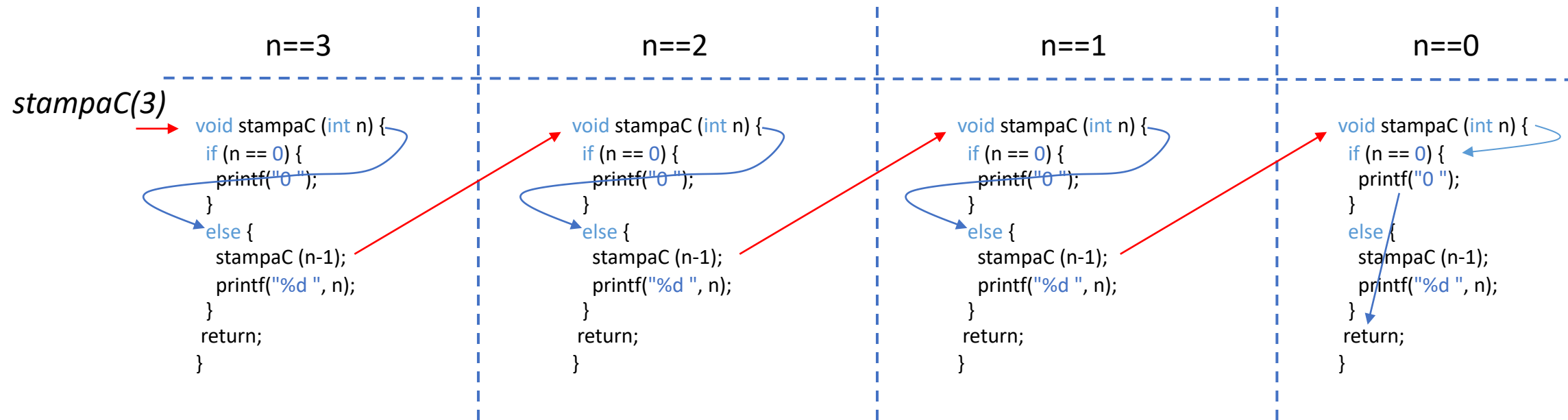


Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N  
0
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione

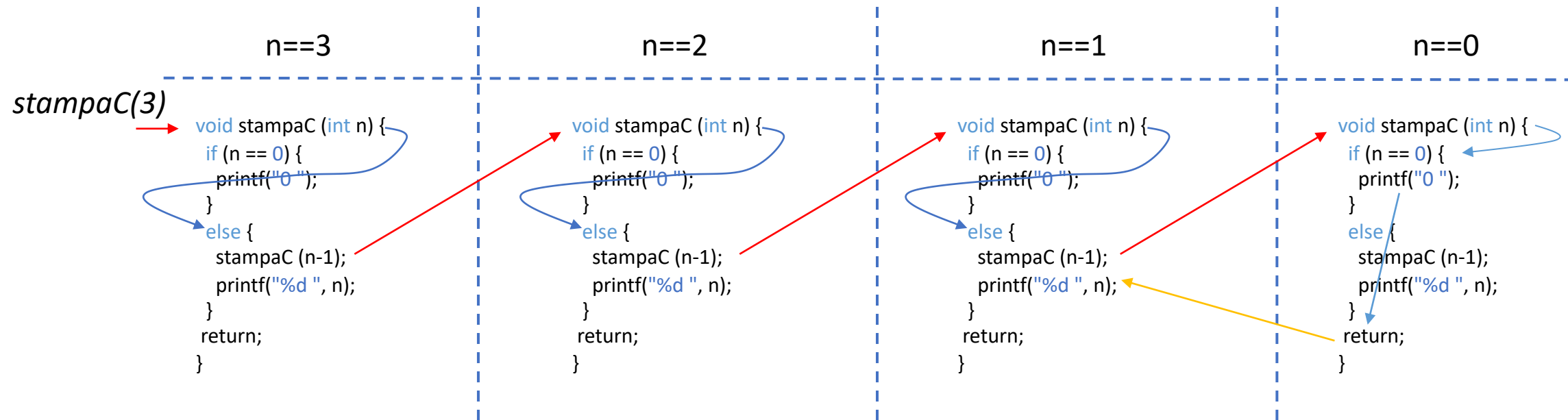


Stampa numeri crescente [0, ..., N]

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione

```
./stampa_ric_0N  
0
```



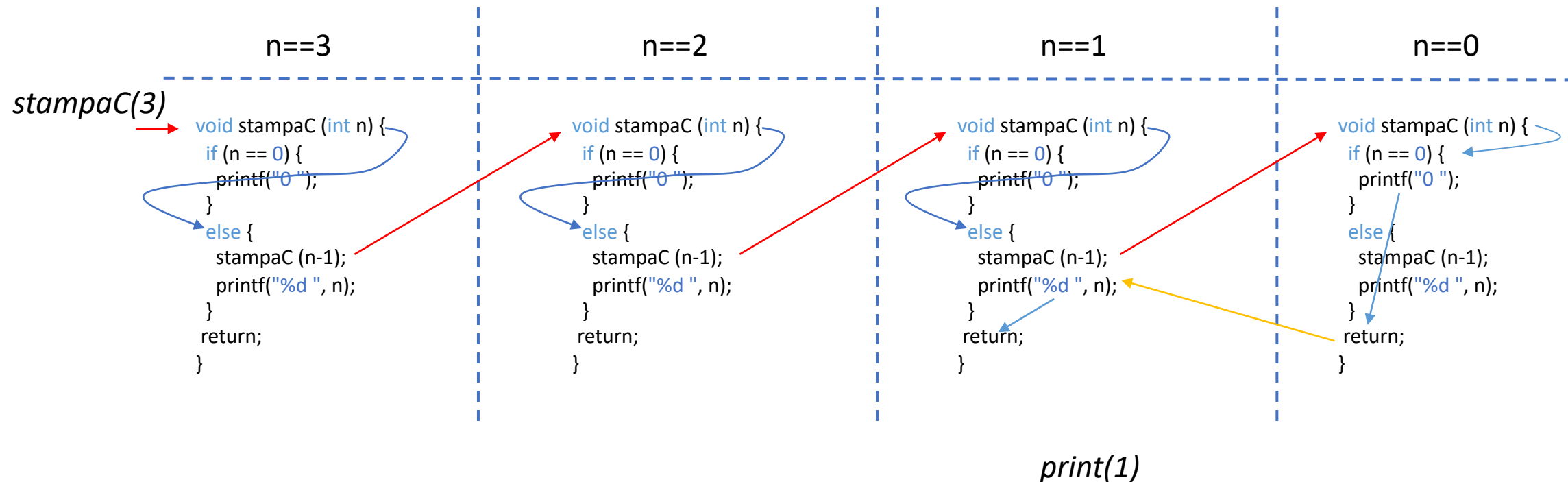
Ritorno a `stampaC(1)`

Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N  
0 1
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione

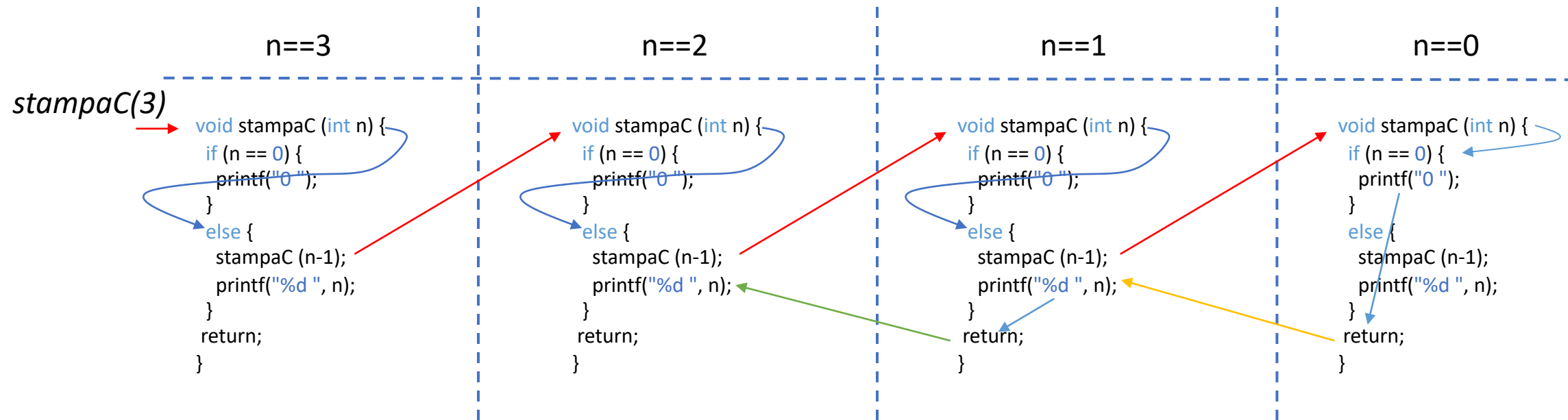


Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N  
0 1
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione



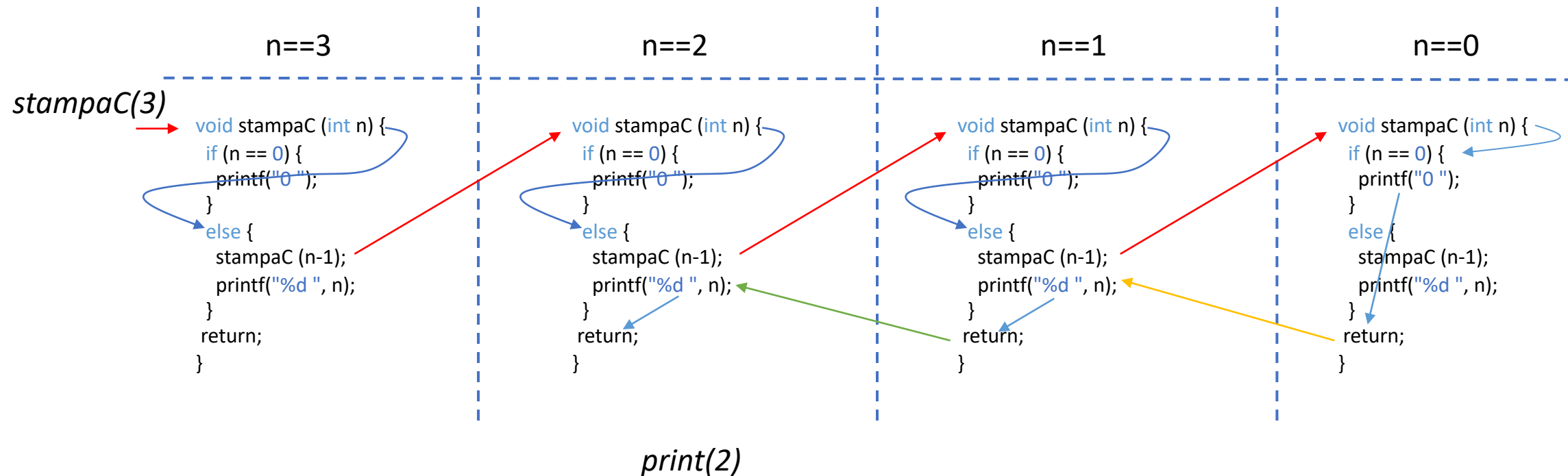
Ritorno a `stampaC(2)`

Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N  
0 1 2
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione

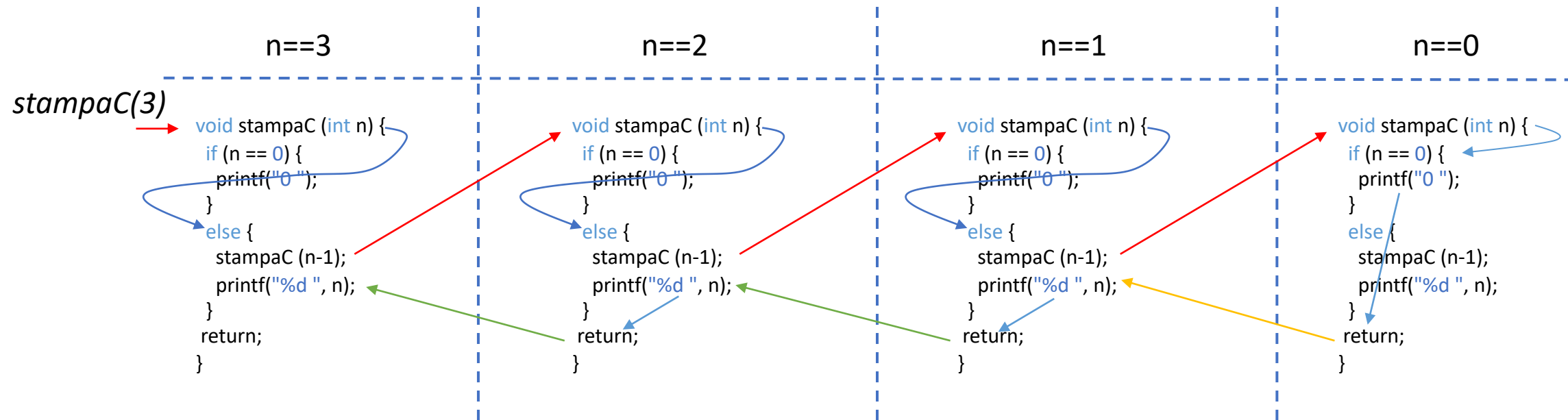


Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N  
0 1 2
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione



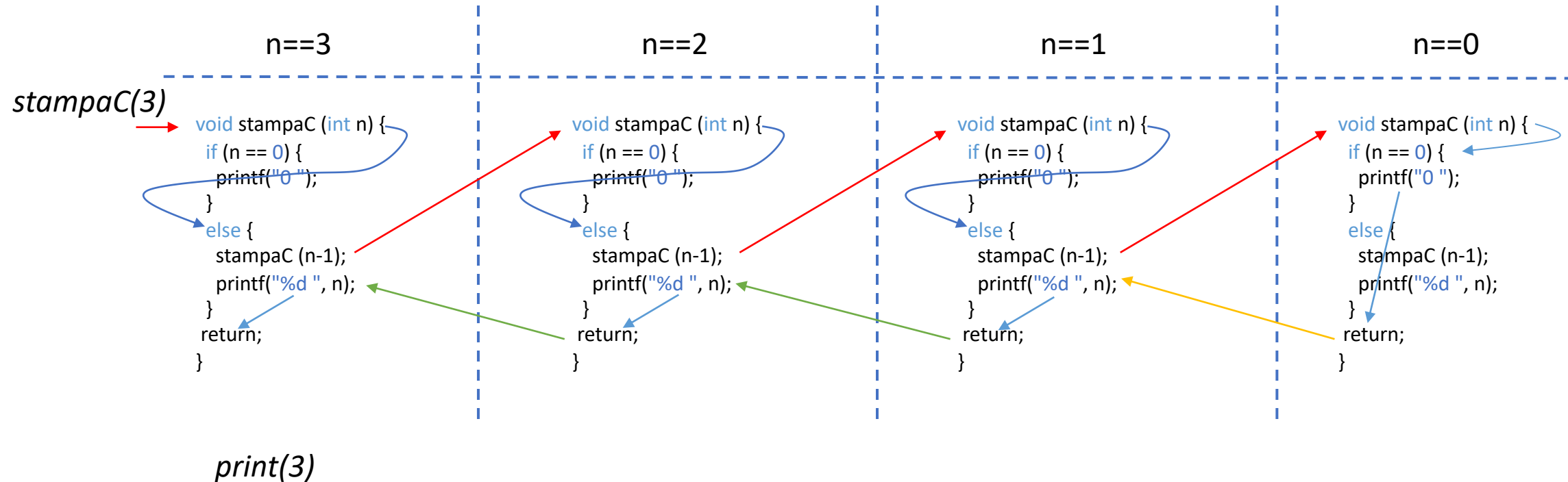
Ritorno a `stampaC(3)`

Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N  
0 1 2 3
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione



Stampa numeri crescente [0, ..., N]

```
./stampa_ric_0N  
0 1 2 3
```

-> chiamata ricorsiva a funzione

<- <- ritorno da chiamata a funzione

